Tutorato 2

Matteo Mazzaretto

27 Ottobre 2026



Distruzioni oggetti

```cpp
class C {
  private:
    string nome;
  public:
    C(const string& n) : nome(n) {
      cout << "Costruito: " << nome << endl;
    }
    ~C() {
      cout << "Distrutto: " << nome << endl;
    }
};

static C ciao("ciao");

C globale("globale");

void funzione() {
    static C statico("statico");
    cout << "Dentro funzione()" << endl;
}

int main() {
    cout << "Inizio main" << endl;
    funzione();
    cout << "Fine main" << endl;
    return 0;
}
```

```
Costruito: ciao
Costruito: globale
Inizio main
Costruito: statico
Dentro funzione()
Fine main
Distrutto: statico
Distrutto: globale
Distrutto: ciao
```

Differenza fra statiche e globali

Entrambe vengono distrutte al termine del programma.
La differenza è dove il loro nome viene “visto” (**linkage**).
Una variabile **static** a livello di file ha linkage interno: è visibile solo nello stesso file.
Una variabile **globale** ha linkage esterno: è visibile in qualsiasi altro file, basta una dichiarazione del tipo

```
extern C globale;
```

Nascondere la parte privata di una classe

Information hiding

Per consegnare una classe C++ bisogna dare l'**header** (`.h`), che contiene sia la parte pubblica sia quella privata: chi lo riceve vede come è fatta la classe dentro.

E se volessimo nascondere la parte privata?

Nell'header lasciamo solo un **puntatore** a una classe "scatola" che contiene i dati privati. La scatola viene **definita** nel `.cpp`, che consegniamo già compilato.

Header (.h)

```
class orario_handle {
  public:
    orario_handle(int o = 0, int m = 0, int s = 0);
    int Ore() const;
    int Minuti() const;
    int Secondi() const;
    void AvanzaUnOra();
  private:
    class orario_rappr;   // dichiarazione incompleta
    orario_rappr* punt;
};
```

Chi legge l'header sa solo che esiste una classe `orario_rappr`, **non** cosa contiene.

Implementazione (.cpp)

```
class orario_handle::orario_rappr {
  public:
    int sec;   // l'orario è salvato in secondi
};

orario_handle::orario_handle(int o, int m, int s): punt(new orario_rappr()) {
  if (...) punt->sec = 0;
  else punt->sec = o*3600 + m*60 + s;
}

int orario_handle::Ore() const {
  return punt->sec / 3600;
}

int orario_handle::Minuti() const {
  return (punt->sec % 3600) / 60;
}

int orario_handle::Secondi() const{
  return punt->sec % 60;
}

void orario_handle::AvanzaUnOra() {
  punt->sec = (punt->sec + 3600) % 86400;
}
```

Problema: condivisione di memoria

Il puntatore è un campo dati, quindi la copia di default copia solo l'**indirizzo**: due oggetti puntano alla **stessa** scatola. Modificarne uno modifica anche l'altro, e quando uno viene distrutto l'altro punta a memoria non valida.

Soluzione: la **regola del 3**, cioè ridefinire

- distruttore
- costruttore di copia
- `operator=`

Regola del 3 applicata

UNIVERSITÀ DI PADOVA

```
orario_handle::~orario_handle() {
  delete punt;
}

orario_handle::orario_handle(const orario_handle& o)
  : punt(new orario_rappr(*o.punt)) {}

orario_handle& orario_handle::operator=(const orario_handle& o) {
  if (this != &o){
    *punt = *o.punt;
  }
  return *this;
}
```

Ogni oggetto ha così la **propria** scatola.

Cosa si consegna: header + compilato

Il venditore compila la sua implementazione:

```
g++ -c orario.cpp      # produce orario.o (codice macchina)
```

e consegna **solo** due file: `orario.h` e `orario.o`. Il sorgente `orario.cpp` resta a lui.

Il compratore scrive `main.cpp` con `#include "orario.h"` e fa:

```
g++ -c main.cpp                    # compilazione: basta l'header
g++ main.o orario.o -o programma # linking: serve il compilato
```

Perché servono entrambi?

- **Compilazione** (`main.cpp` ⟶ `main.o`): il compilatore legge l'header per sapere che classi e metodi esistono e come si chiamano. Non gli servono i corpi dei metodi.
- **Linking**: le chiamate come `t.Ore()` sono rimaste "in sospeso"; il linker le collega al codice che si trova in `orario.o`.

L'header dice *che cosa* esiste, il file compilato contiene *come* funziona.

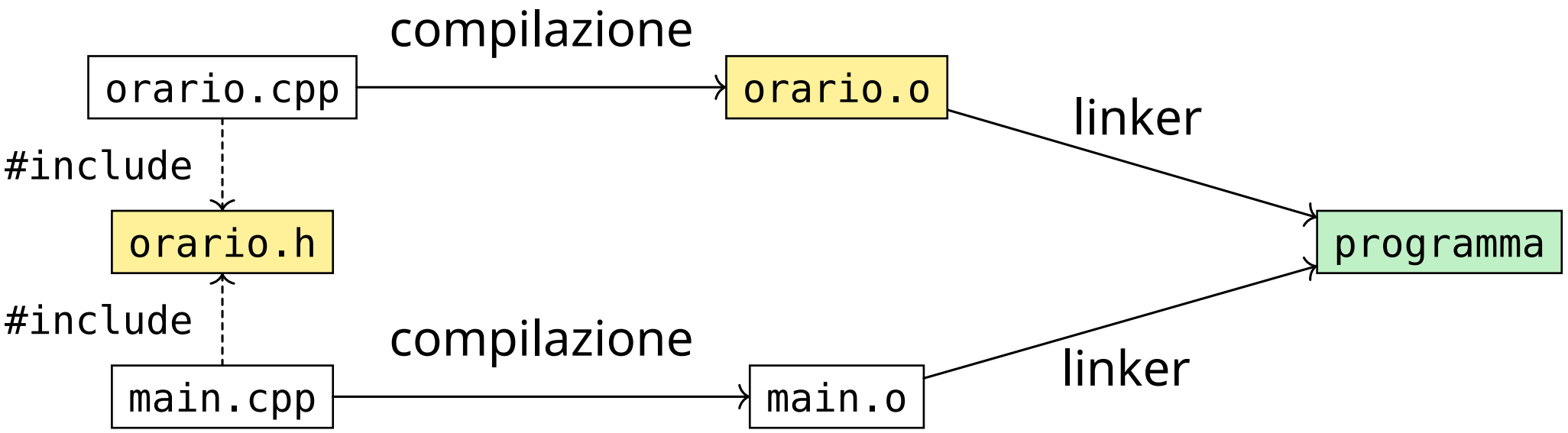
Perché serve il puntatore

Per creare un oggetto il compilatore deve conoscerne la **dimensione**.
In una classe normale deve quindi vedere tutti i campi `private`: per questo vanno nell'header.

Con il puntatore a una classe **incompleta** la dimensione è solo quella di un puntatore: il compilatore non ha bisogno di sapere cosa c'è dentro `orario_rappr`.

La definizione di `orario_rappr` sta solo nel `.cpp`, quindi solo in `orario.o`.

Dal sorgente al programma



In giallo i file che il venditore **consegna**. `orario.cpp` resta a lui. Le frecce tratteggiate sono gli `#include`: l'header serve al compilatore, `orario.o` serve al linker.

Array di oggetti

Costruzione e distruzione

Quando si crea un array (statico o dinamico), il **costruttore di default** viene chiamato implicitamente **una volta per ogni elemento**.
Se la classe non ha un costruttore di default, l'array non si può creare.

Il **distruttore** viene chiamato per ogni elemento:

- array **statico**: alla fine dello scope in cui è dichiarato
- array **dinamico**: solo quando si scrive `delete[]`

ATTENZIONE: un array creato con `new[]` va distrutto con `delete[]` (non `delete`), altrimenti i distruttori non vengono chiamati.

Esempio di cosa stampa

```
class C {
  public:
    int i;
    C(int x = 3) : i(x) { cout << "C01 "; }
    ~C() { cout << "D01 "; }
};

int main() {
  C a[4];              // stampa: C01 C01 C01 C01
  C* ad = new C[2];    // stampa: C01 C01
  for (int j = 0; j < 4; j++)
    cout << a[j].i;          // stampa: 3333
  for (int j = 0; j < 2; j++)
    cout << (ad+j)->i;       // stampa: 33
  delete[] ad;         // stampa: D01 D01
}                      // fine main: a[] distrutto, stampa D01 x4
```

Casting

Tipologie di cast

1. Explicit cast (ereditato dal C)
2. Static cast
3. Const cast
4. Reinterpret cast
5. Dynamic cast

Explicit cast

Viene utilizzata la notazione “( )”.

```
float x = 3.5;
char c = 'a';
int a = 5;
void* p = &a;

int i = (int) x;       // i = 3
float y = (float) c;   // y = 97 (codice ASCII di 'a')
int* q = (int*) p;     // q punta allo stesso indirizzo di p
```

ATTENZIONE: il cast stile C prova in sequenza `static_cast`, `const_cast` e `reinterpret_cast`, quindi può nascondere conversioni pericolose.

Static cast

Lo static cast rende **esplicite** le conversioni implicite e permette anche alcune conversioni inverse (ad esempio `double` → `int`, `void*` → `T*`).
Si basa sul tipo statico, cioè noto a compile-time (quando si fa `g++ -c main.cpp`).
Il warning per la perdita di informazione non compare, perché la conversione è stata richiesta esplicitamente.
Le seguenti conversioni implicite sono senza perdite d'informazione:

1. T& → T
2. T[] → T*
3. T → const T
4. T* → const T*
5. float → double → long double
6. char → short → int → long
7. unsigned char → ... → unsigned long

```
float x = 3.5;
int i = static_cast<int>(x);   // i = 3
```

Const cast

Permette di rimuovere il “const” da un **puntatore o riferimento**.
Non andrebbe usato quasi mai, perché viola il principio di “constness”.
Se l’oggetto originale è **dichiarato** `const`, modificarlo è comportamento indefinito.

```
int y = 3;
const int& r = y;
r = 5;                          // errore
const_cast<int&>(r) = 5;  // compila
```

Reinterpret cast

Il più pericoloso dei cast: reinterpreta la sequenza di bit come se fosse di un altro tipo, senza nessuna conversione.
Va usato solo con estrema cautela.
Il risultato dipende dall'architettura.

```
double d = 3.14;
int* i = reinterpret_cast<int*>(&d);
cout << *i; // legge i primi 4 byte di d come int
```

Dynamic cast

A differenza dello `static_cast`, il controllo avviene a **run-time**.
Si usa con puntatori o riferimenti a classi in una gerarchia di ereditarietà (argomento che verrà coperto in futuro).
Se la conversione non è valida, restituisce `nullptr` (per i puntatori).

Friend

Funzioni amiche

Una funzione esterna alla classe può usare solo l'interfaccia **pubblica**.
Nell'esempio di bolletta, `operator<<` riceve la bolletta **per valore** (una copia) e la "svuota" con `Estrai_Una`:

```
ostream& operator<<(ostream& os, bolletta b) {
  int i = 1;
  while (!b.Vuota()) {
    os << i << ") " << b.Estrai_Una() << endl;
    i++;
  }
  return os;
}
```

Dichiarare la funzione friend

Dichiarando la funzione `friend` dentro la classe, può accedere alla parte privata e lavorare direttamente sull'oggetto, senza copie.

```
class bolletta {
  ......
  friend ostream& operator<<(ostream&, const bolletta&);
};

ostream& operator<<(ostream& os, const bolletta& b) {
  bolletta::nodo* p = b.first;
  int i = 1;
  while (p) {
    os << i << ") " << p->info << endl;
    i++;
    p = p->next;
  }
  return os;
}
```

Iteratori

Iteratori applicati a Instagram

Per scorrere la lista di `Reel` senza mostrare i nodi all'esterno, aggiungiamo a `Instagram` una classe annidata **iteratore**, che incapsula un puntatore a `Reel`.

L'utente usa `begin()`, `end()`, `++` e `[]` come se fossero indici, senza sapere che dietro c'è una lista.

Classe Instagram con iteratore

UNIVERSITÀ DI PADOVA

```
class Instagram {
  private:
    class Reel {
      public:
        int sec;
        Reel* next;
        Reel(int s, Reel* n) : sec(s), next(n) {}
    };
    Reel* first;
  public:
    class iteratore {
      friend class Instagram;
      private:
        Reel* punt;
      public:
        iteratore() : punt(nullptr) {}
        bool operator==(iteratore i) const {
          return punt == i.punt;
        }
        bool operator!=(iteratore i) const {
          return punt != i.punt;
        }
        iteratore& operator++() {     // prefisso
          if (punt) punt = punt->next;
          return *this;
        }
        iteratore operator++(int) {   // postfisso (int solo per distinguerlo)
          iteratore aux = *this;
          if (punt) punt = punt->next;
          return aux;
        }
    };
```

Instagram: begin, end e operator[]

```
    Instagram() : first(nullptr) {}

    void aggiungi_in_testa(int s) {
      first = new Reel(s, first);
    }

    iteratore begin() const {
      iteratore aux;
      aux.punt = first;
      return aux;
    }

    iteratore end() const {
      iteratore aux;
      aux.punt = nullptr;   // "oltre l'ultimo"
      return aux;
    }

    int& operator[](iteratore it) const {
      return it.punt->sec;
    }

    bool operator==(const Instagram& x) const;
    bool operator!=(const Instagram& x) const;
};
```

`end()` non punta a un `Reel` valido: indica che la lista è finita.

Instagram: == e != con gli iteratori

Due `Instagram` sono uguali se hanno gli stessi `sec`, nello stesso ordine. Scorriamo le due liste insieme con gli iteratori:

```
bool Instagram::operator==(const Instagram& x) const {
  iteratore i = begin();
  iteratore j = x.begin();
  while (i != end() && j != x.end()) {
    if ((*this)[i] != x[j])
      return false;
    ++i;
    ++j;
  }
  return i == end() && j == x.end();
}

bool Instagram::operator!=(const Instagram& x) const {
  return !(*this == x);
}
```

Alla fine del ciclo le liste sono uguali solo se **entrambe** sono terminate: se una è più lunga, sono diverse.

Uso: somma delle durate

```
int Somma_Durate(const Instagram& x) {
  int durata = 0;
  for (Instagram::iteratore it = x.begin(); it != x.end(); ++it) {
    durata += x[it];
  }
  return durata;
}
```

Chi usa `Instagram` non vede né `Reel` né i puntatori: sa solo scorrere con l'iteratore.

Nel `for` conviene `++it` (prefisso): il postfisso crea una copia in più che non serve.